[PRD] 내부 개발 포털 - 커스텀 관리도구 구현 명세 (UI & Flow)

🗒️ 히스토리

날짜	내용	작성자
2026년 3월 9일	draft v.0.1. • 디자인 demo3.pen	@서상은/mena
2026년 3월 10일	v.0.2. 준케이 싱크업 후 업데이트	@서상은/mena

🎯 Phase 1 구현 범위

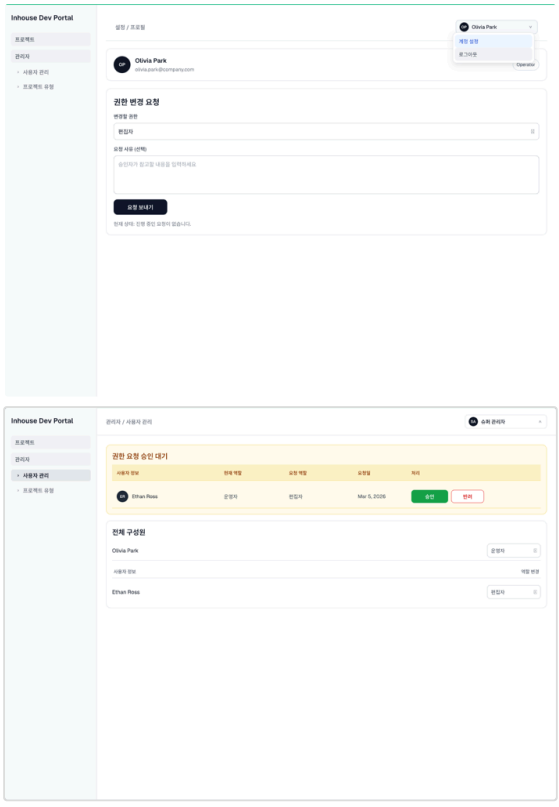
- 일정: 3월 런칭
- 우선순위
  a. Liquid.js 라우팅 자동화 및 인프라 연동 구현
  b. 사용자가 커스텀 프로젝트를 설정할때 필수 스펙 구현
  c. 사용자가 추후 내부 개발 포털을 사용할 때 optional 스펙 구현

0. 기본 셋팅

1	GitHub Personal Access Token (PAT)	포털 화면에서 사용자가 `[프로젝트 생성]` 을 누르면, 백엔드가 GitHub API를 찔러서 `partners-bmf` 조직 안에 새로운 레포지토리를 자동으로 만드는데 설정
2	Webhook(웹훅) 수신 엔드포인트 & 시크릿 키	• GitHub Actions에서 배포가 끝났을 때(성공/실패), 우리 포털 백엔드로 "배포 끝났어!"라고 알려주는 용도
3	포털 DB 구축 방식 및 DB 접속 정보	• 현 진행 ◦ 데이터베이스는 나중에 회사 DB 이전할 예정이지만, 지금은 로컬 환경에서 빠르게 개발하기 위해 'Prisma ORM'과 'SQLite'를 세팅해서 진행 • 나중에는 어떻게 해야하는지? ◦ 프로젝트 메타데이터, 노출 On/Off 상태를 저장할 PostgreSQL/MySQL DB 주소(URL)가 필요 ◦ 회사의 운영 DB에 포털용 DB를 올려야하지 않는지? ◦ devops에 DB 셋팅요청? dev/qa/real
4	포털 자체의 도메인 (Hosting)	• admin-custom.astage.in • 호스팅은 가이드 문서 참고하여 진행?
5	Cloudfront	캐시 무효화 처리 - 필요 시 셋팅 요청 • AWS에 접근할 수 있도록 1) CloudFront Distribution ID, 그리고 2) cloudfront:CreateInvalidation 권한이 있는 AWS Access Key / Secret Key (또는 IAM Role) 필요

6	깃헙	bmf organization 내 포털용 깃헙 레포지토리 등록하여 진행

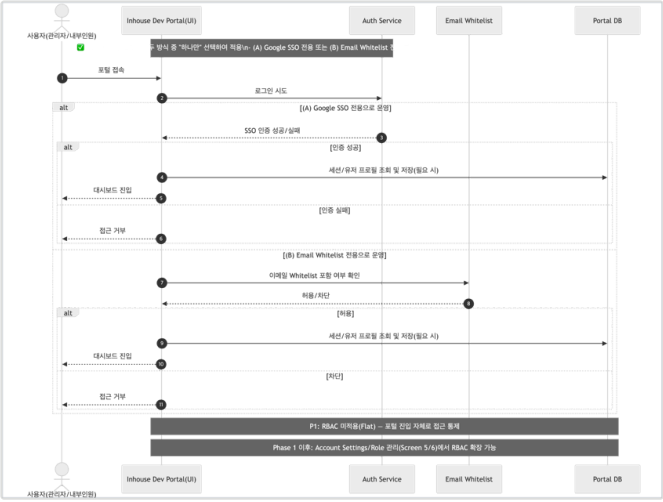
1. 접속 및 인증 (Authentication & Access)

디자인	
로그인	
권한	

1.1. 스펙

	기능	설명	우선순위	비고
1	로그인 기능	• 1안. 사내 구글 SSO 로그인 또는 • ~~2안. 사전에 지정된 이메일 (Whitelist) 계정만 접근할 수 있도록 차단~~ • ~~둘중에 뭘로할지?~~	상	
2	권한 제어 (RBAC)	• P1에서는 내부 개발 포털에 진입할 수 있는 인원이 이미 통제되어 있으므로, 사용자별 프로젝트 생성 권한이나 복잡한 역할 기반 접근 제어(RBAC)는 적용하지 않고 평면적(Flat)인 권한 구조로 운영 • phase 1 이후 적용	하	

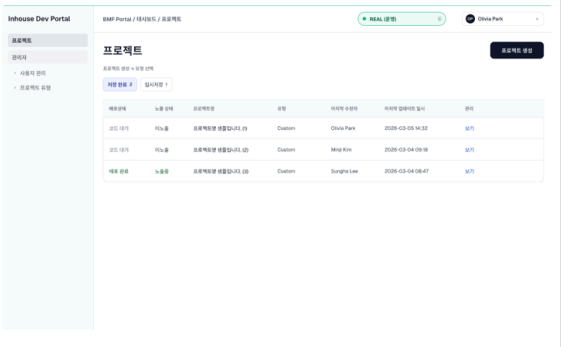
1.2. 사용자 접속 순서



2. 프로젝트 생성 및 기본 정보 입력 (Project Creation)

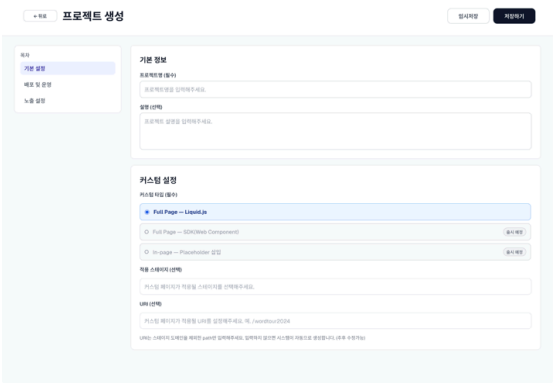
디자인

프로젝트 목록



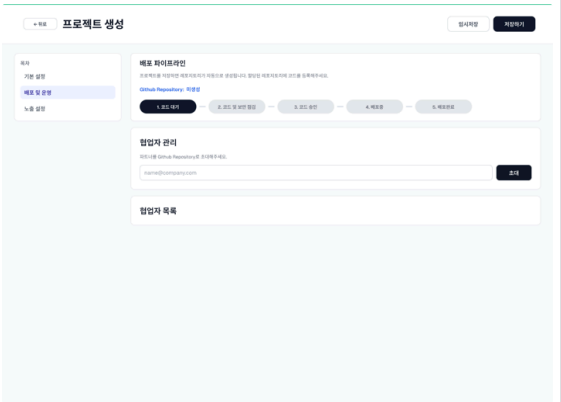
프로젝트 생성

기본 설정



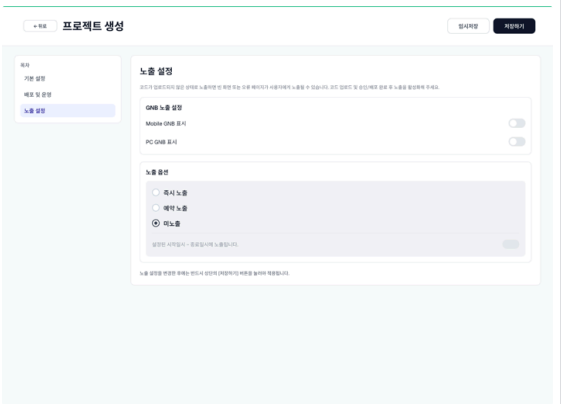
프로젝트 생성

배포 및 운영

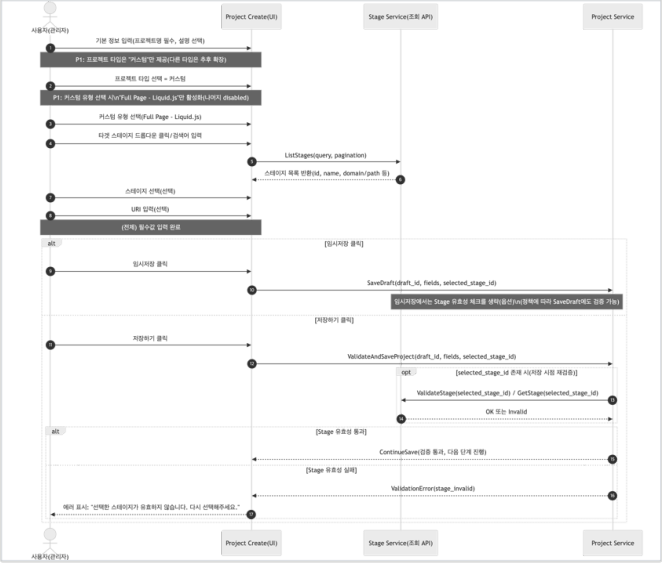


프로젝트 생성

노출 설정

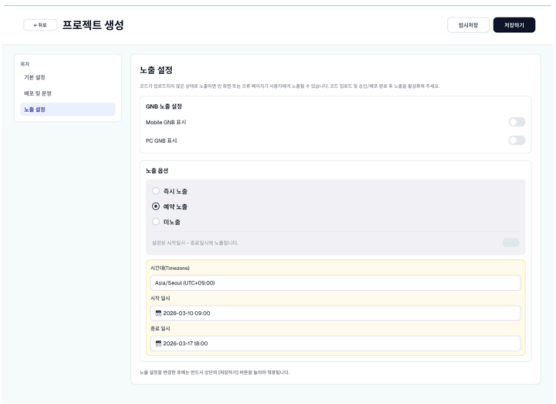


2.1. 프로젝트 생성 시퀀스

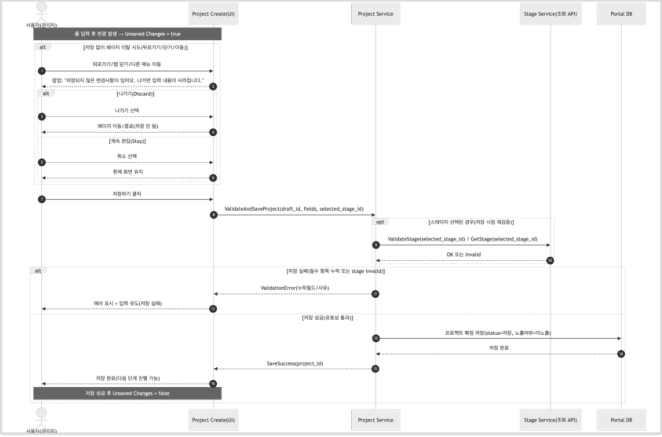


2.2. 스펙

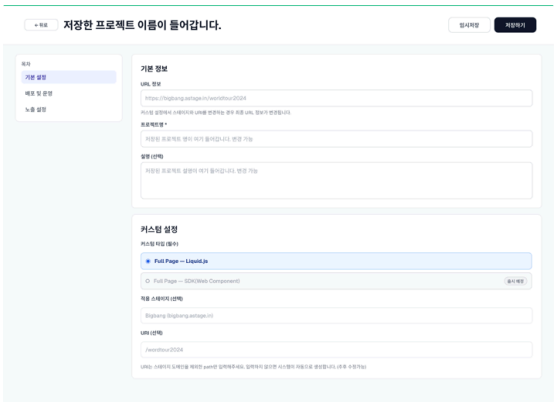
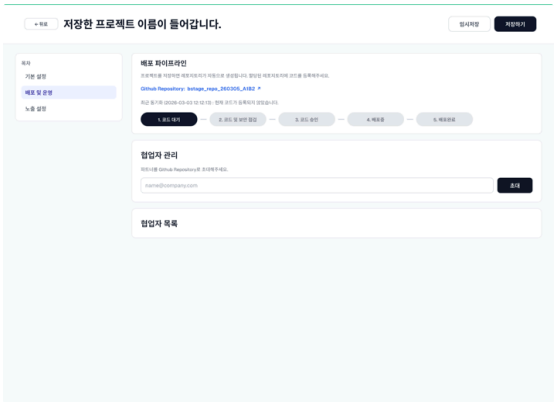
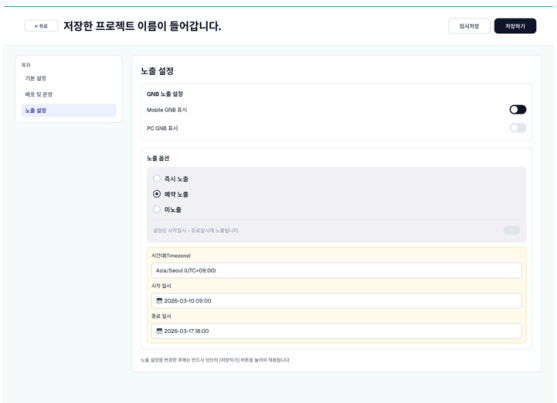
	기능	설명	우선순위	비고
1	프로젝트 목록	• N개 프로젝트 생성 가능	상	
2	프로젝트 타입	• N개 프로젝트 타입 가능 (ex. 커스텀, 검색..) ◦ 슈퍼 관리자가 생성 • P1에서는 커스텀 프로젝트로 생성으로 고정	하 (구조만 마련)	
3	저장	• 임시저장	하	
4		• 저장하기 • 필수항목 및 유효한 형식 밸리데이션 • 화면) 스피너 + 저장중입니다 공통으로 하나 만들기	상	
5	프로젝트 메타 정보	• 1) 프로젝트 명(필수) • 2) 프로젝트 설명(선택)	중	
6	커스텀 타입	• 필수값 • Full page- liquid.js 상 • Full page - SDK P1 제외 • In page P1 제외	좌측 참고	
7	연결 스테이지	• 필수값 (URL생성에 필요) • 스테이지 조회 api 사용 - **확인 필요** • 저장 시점에 Stage Service(조회 API)를 다시 호출해서 스테이지 유효성 검증		
8	URI	• 유저 인풋할지 여부는 선택	중	

예약노출				
		◦ 미입력하는 경우 시스템 자동발급 ◦ CloudFront에 연결된 정식 도메인 (https://dev.bstage.in/custom/...)으로만 발급 • 동일한 개발환경 및 스테이지에서 중복 불가		
9	협업자 관리 및 목록	• 외부 파트너 초대용이나 phase 1에서는 구현하지 않음 • 프로젝트 저장 후 & 레포 생성 완료된 경우에만 활성화	P2 이후	
10	GNB 노출 설정	• 연결된 스테이지의 GNB 노출 ◦ 기본값: 미노출 ◦ PC ◦ 모바일 • **연결된 스테이지의 GNB 조회 API 필요**	중	
11	노출 옵션	• 즉시노출 • 미노출 (기본값)	상	
12		예약 노출 • 타임존 • 시작일시 • 종료일시 • 시작일시 <= 종료일시 유효성 체크	하	

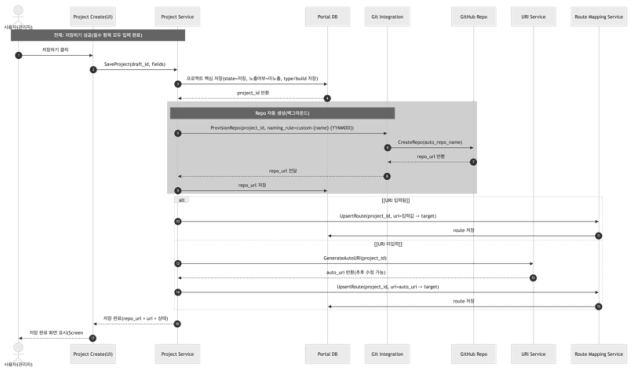
2.2. 저장 밸리데이션 분기 (저장 실패 vs 저장 성공)



3. 프로젝트 저장에 따른 깃헙 Repo 및 URI 자동 생성

디자인	
기본설정	
배포 및 운영	
노출 설정	

3.1. GitHub 레포 자동 생성 및 URI 발급 시퀀스



GitHub 레포 자동 생성 및 URI 발급 시퀀스

```
sequenceDiagram
    autonumber
    actor U as 사용자(관리자)
    participant UI as Project Create(UI)
    participant PS as Project Service
    participant DB as Portal DB
    participant GI as Git Integration
    participant GH as GitHub Repo
    participant URI as URI Service
    participant RM as Route Mapping Service

    Note over U,PS: 전제: 저장하기 성공(필수 항목 모두 입력 완료)

    U->>UI: 저장하기 클릭
    UI->>PS: SaveProject(draft_id, fields)
    PS->>DB: 프로젝트 핵심 저장(state=저장, 노출여부=미노출, type/b
    DB-->>PS: project_id 반환

    rect rgba(120,120,120,0.35)
        Note over PS,GI: Repo 자동 생성(백그라운드)
        PS->>GI: ProvisionRepo(project_id, naming_rule=custom
        GI->>GH: CreateRepo(auto_repo_name)
        GH-->>GI: repo_url 반환
        GI-->>PS: repo_url 전달
        PS->>DB: repo_url 저장
    end

    alt [URI 입력됨]
        PS->>RM: UpsertRoute(project_id, uri=입력값 -> target)
        RM->>DB: route 저장
    else [URI 미입력]
        PS->>URI: GenerateAutoURI(project_id)
        URI-->>PS: auto_uri 반환(추후 수정 가능)
        PS->>RM: UpsertRoute(project_id, uri=auto_uri -> targ
        RM->>DB: route 저장
    end

    PS-->>UI: 저장 완료(repo_url + uri + 상태)
    UI-->>U: 저장 완료 화면 표시(Screen #1)
```

3.2. 스펙

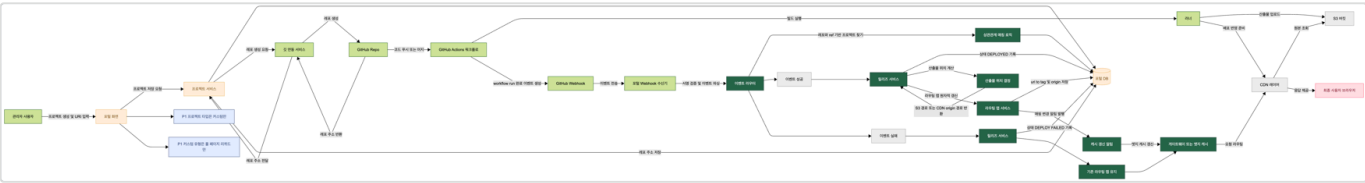
	기능	설명	우선순위	비고
1	저장하기	• 캐시정책 ◦ dev/qa: 캐시 없음 ◦ real: 현 정책 - 캐시 1시간 적용됨 • 캐시 날리는 강제 새로고침 필요 → 저장하기 클릭 시 ◦ 1) AWS SDK(v3)를 사용해서 S3 버킷에 라우팅 정보(routes.json)를 덮어쓰기. ◦ 2) S3 업로드가 성공하면, 즉시 AWS SDK를 사용해 CloudFront의 CreateInvalidationCommand를 호		

		출해서 /* (모든 경로)에 대해 캐시 무효화 요청 ◦ 2번 관련 화면) 스피너 + 서버에 반영중입니다 나오도록		
2	정보 속성	• 프로젝트명: 변경가능, 중복가능 • 프로젝트 설명: 변경가능, 중복가능 • 프로젝트 ID: 변경불가, 중복불가 (unique)- 시스템 자동 채번 • 프로젝트 타입: 변경불가 --- • 커스텀 타입: 변경 불가 ◦ Full page ↔ In page 변경 불가 (개발방식 상이) ◦ 똑같은 URI에 뒷단의 파일만 바뀌는 경우 ◦ 1) 레포 자체가 바뀌는 경우 ▪ 가. 외주 제작사(Agency)가 변경되었을 때 (보안/계약 문제) ▪ 나. 검색엔진(SEO) 마케팅 자산을 재활용 ▪ 다. liquid.js ↔ SDK 변경시 – 이때는 새로운 프로젝트를 만들고, 프로젝트의 노출설정을 통해 제어하도록 가이드. ◦ 2) 동일한 레포에 소스가 바뀌는 경우 ▪ tag로 버전관리 ▪ 자동 배포되도록 특정 예약시간에 버전이 바뀌어서 보여야 하는 케이스 • 깃헙에 tag가 여러 버전으로 존재하고 • 포털 화면에는 tag 버전 조회, 선택 → 선택시 어떤 버전을 배포하라는 트리거를 깃헙으로 전송 → 자동배포되도록 진행 --- • 스테이지: 저장 후 변경 불가 (URL에 영향) --- • URI ◦ 배포 완료시 변경 불가 ◦ 배포 완료 상태가 아니고 && 프로젝트 노출상태 = 미노출이면 변경 가능 ◦ 동일한 개발환경 & 동일한 스테이지에서 중복 불가	중	
3	프로젝트 삭제	• 프로젝트 삭제 가능,	상	

		◦ 단, 포털DB에서 실제 삭제가 아니라 여러가지 엮어진 정보가 많기 때문에(오래걸리고, 맘바뀔수도있고) deleted flag로 하고 저장된상태, 화면엔 안보이도록 처리		
4	Github repository 자동 생성	• 프로젝트 저장 완료 시 해당 프로젝트 전용 레포지토리가 자동 생성됨 • 프로젝트: 레포지토리 = 1:1 • 레포지토리 네이밍 규칙 ◦ devops에서 vendor, space 등등.. 필요한지 확인필요 ◦ `https://cdn.static.bstage.in/3pp-templates/{vendor}/user/{space}/{template}/images/logo.png` ◦ `bstage-bemyfriends-templates` ◦ {프로젝트타입}-{프로젝트ID}-{최초생성일자YYMMDD(n개 생성시 인지할수있도록..thinking..} • 참고) 레포지토리 생성 문서 리포지토리에 대한 REST API 엔드포인트 - GitHub 문서	상	
5	배포 상태	1.4.에 후술	중	
6	자동 배포	1.4.에 후술	상	
7	협업자 관리 및 목록	• 외부 파트너 초대용이나 phase 1에서는 구현하지 않음 • 프로젝트 저장 후 & 레포 생성완료된 경우에만 활성화	P2 이후	

4. 배포 및 운영

- **자동 배포 파이프라인:** 레포지토리에 코드가 정상적으로 승인/반영되면, 백그라운드에서 CI/CD 시스템(Runner)이 자동으로 돌아감. Runner가 코드를 빌드하여 S3 버킷(창고)과 CDN에 최종 결과물을 진열합니다.
  - 참고. Liquid Template Deployment Flow Overview
- **배포 현황 모니터링:** 사용자는 포털 화면에서 자동 생성된 **GitHub의 Tag(버전) 값**과 **배포 진행 상태**를 직관적으로 확인할 수 있습니다.
- **라우팅:** 배포가 '완료'된 후, 설정한 서비스 주소와 S3/CDN 결과물이 자동 매핑되어 실제 화면이 노출됩니다



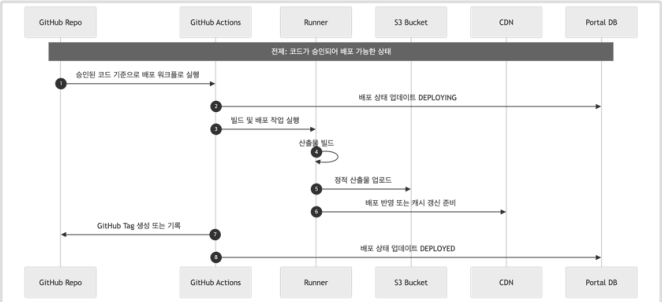
배포 및 운영 흐름 시퀀스

```
---
config:
  layout: dagre
  theme: forest
  look: classic
---
flowchart LR
    U["관리자 사용자"] -- 프로젝트 생성 및 URI 입력 --> UI["포털 화면"]
    UI -- 프로젝트 저장 요청 --> PS["프로젝트 서비스"]
    PS --> DB[("포털 DB")]
    PS -- 레포 생성 요청 --> GI["깃 연동 서비스"]
    GI -- 레포 생성 --> GH["GitHub Repo"]
    GH -- 레포 주소 반환 --> GI
    GI -- 레포 주소 전달 --> PS
    PS -- 레포 주소 저장 --> DB
    GH -- 코드 푸시 또는 머지 --> ACT["GitHub Actions 워크플로"]
    ACT -- 빌드 실행 --> RUN["러너"]
    RUN -- 산출물 업로드 --> S3["S3 버킷"]
    RUN -- 배포 반영 준비 --> CDN["CDN 레이어"]
    ACT -- workflow run 완료 이벤트 생성 --> GHWH["GitHub Webhook"]
    GHWH -- 이벤트 전송 --> WHR["포털 Webhook 수신기"]
    WHR -- 서명 검증 및 이벤트 파싱 --> EVT["이벤트 라우터"]
    EVT -- 레포와 ref 기반 프로젝트 찾기 --> CORR["상관관계 매칭 로직"]
    CORR --> DB
    EVT --> OK["이벤트 성공"] & NG["이벤트 실패"]
    OK --> REL1["릴리즈 서비스"]
    REL1 -- 상태 DEPLOYED 기록 --> DB
    REL1 -- 산출물 위치 계산 --> ART["산출물 위치 결정"]
    ART -- S3 경로 또는 CDN origin 경로 반환 --> REL1
    REL1 -- 라우팅 맵 원자적 갱신 --> RM["라우팅 맵 서비스"]
    RM -- uri to tag 및 origin 저장 --> DB
    RM -- 매핑 변경 알림 발행 --> BUS["캐시 갱신 알림"]
    BUS -- 엣지 캐시 갱신 --> EDGE["게이트웨이 또는 엣지 캐시"]
    EDGE -- 요청 라우팅 --> CDN
    CDN -- 원본 조회 --> S3
    CDN -- 응답 제공 --> END["최종 사용자 브라우저"]
    NG --> REL2["릴리즈 서비스"]
    REL2 -- 상태 DEPLOY FAILED 기록 --> DB
    REL2 --> KEEP["기존 라우팅 맵 유지"]
    KEEP --> EDGE
    UI --> N1["P1 프로젝트 타입은 커스텀만"] & N2["P1 커스텀 유형은 풀 페이

     UI:::Peach
     PS:::Peach
     DB:::Peach
     S3:::Ash
     CDN:::Ash
     EVT:::Pine
     CORR:::Pine
     OK:::Ash
     NG:::Ash
     REL1:::Pine
     ART:::Pine
     RM:::Pine
     BUS:::Pine
     EDGE:::Pine
     END:::Rose
     REL2:::Pine
     KEEP:::Pine
     N1:::Sky
     N2:::Sky
    classDef Sky stroke-width:1px, stroke-dasharray:none, stroke:#37
    classDef Peach stroke-width:1px, stroke-dasharray:none, stroke:#
    classDef Aqua stroke-width:1px, stroke-dasharray:none, stroke:#4
    classDef Pine stroke-width:1px, stroke-dasharray:none, stroke:#2
    classDef Ash stroke-width:1px, stroke-dasharray:none, stroke:#99
    classDef Rose stroke-width:1px, stroke-dasharray:none, stroke:#F
```

4.1. 자동 배포 파이프라인

- 승인된 코드가 기준이 되면 **GitHub Actions**가 자동 실행 상
- **Runner**
  - 빌드 후 **S3**에 산출물 업로드 상
  - **CDN** 반영 준비까지 포함 상
  - 배포 완료시, 배포된 tag 버전 상
    - 배포 실패 시에는 라우팅 맵을 변경하지 않고, 기존에 서비스 중이던 버전 매핑을 그대로 유지
- <u>**확인할 사항**</u>
  - 배포가 실패했을때, 뭔가 강제 다시 배포시작하는 기능 필요. 기능에 추가 상
  - 킬 스위치
    - 어떤 걸 킬 시킬지 체크
    - 롤백 기능 - 사용자는 포털 > 프로젝트 상세에서 태그 버전 골라서 배포시킬 수 있다.

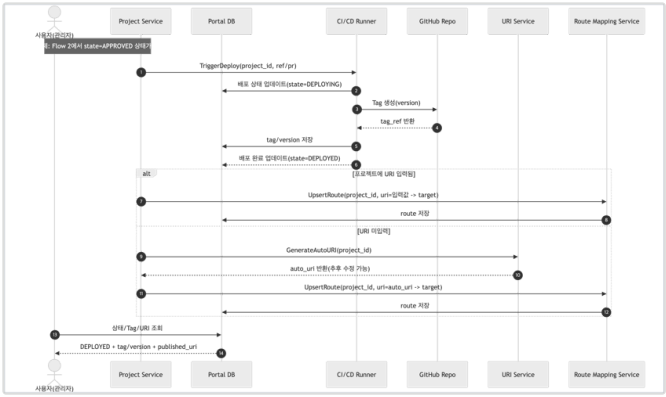
4.1.1. 자동 배포 시퀀스



자동 배포 시퀀스

```
sequenceDiagram
    autonumber
    participant GH as GitHub Repo
    participant GA as GitHub Actions
    participant R as Runner
    participant S3 as S3 Bucket
    participant CDN as CDN
    participant DB as Portal DB

    Note over GH,DB: 전제: 코드가 승인되어 배포 가능한 상태

    GH->>GA: 승인된 코드 기준으로 배포 워크플로 실행
    GA->>DB: 배포 상태 업데이트\nDEPLOYING
    GA->>R: 빌드 및 배포 작업 실행
    activate R
    R->>R: 산출물 빌드
    R->>S3: 정적 산출물 업로드
    R->>CDN: 배포 반영 또는 캐시 갱신 준비
    deactivate R
    GA->>GH: GitHub Tag 생성 또는 기록
    GA->>DB: 배포 상태 업데이트\nDEPLOYED
```

4.1.2. Github tag 발급 시퀀스

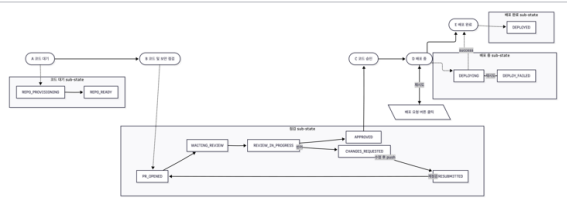


Github tag 발급 시퀀스

```
sequenceDiagram
    autonumber
    actor U as 사용자(관리자)
    participant PS as Project Service
    participant DB as Portal DB
    participant CI as CI/CD Runner
    participant GH as GitHub Repo
    participant URI as URI Service
    participant RM as Route Mapping Service

    Note over U,PS: 전제: Flow 2에서 state=APPROVED 상태가 됨

    PS->>CI: TriggerDeploy(project_id, ref/pr)
    CI->>DB: 배포 상태 업데이트(state=DEPLOYING)
    CI->>GH: Tag 생성(version)
    GH-->>CI: tag_ref 반환
    CI->>DB: tag/version 저장
    CI->>DB: 배포 완료 업데이트(state=DEPLOYED)

    alt 프로젝트에 URI 입력됨
        PS->>RM: UpsertRoute(project_id, uri=입력값 -> target)
        RM->>DB: route 저장
    else URI 미입력
        PS->>URI: GenerateAutoURI(project_id)
        URI-->>PS: auto_uri 반환(추후 수정 가능)
        PS->>RM: UpsertRoute(project_id, uri=auto_uri -> targ
        RM->>DB: route 저장
    end

    U->>PS: 상태 / Tag / URI 조회
    PS->>DB: 상태 / Tag / URI 조회
    DB-->>PS: DEPLOYED + tag/version + published_uri
    PS-->>U: DEPLOYED + tag/version + published_uri
```

4.2. 배포 현황 모니터링

- **확인할 내용**
  - **웹훅 연동:** 우리 사내망(방화벽) 환경에서 깃허브가 보내는 이 알람(웹훅)을 포털 시스템이 잘 받을 수 있게 통신이 뚫려있는 것으로 확인 완료
  - **배포 주체:** 저희 커스텀 코드(Liquid.js) 실제 배포는 깃허브 액션(GitHub Actions)이 끝까지 다 처리하고 있으므로, 깃헙에서 웹훅 받으면 됨.
  - **완료 기준:** 포털에서 '배포 완료' 표기하는 시점은, 파일과 URI 매핑이 완료되어서 (라우트맵 업데이트) 사인을 줄때 시점으로 한다.

배포상태에 따른 상태값 변화	

- 포털 > 프로젝트 상세에서 각 sub 상태키별 안내 문구제공 중
  - 최근 동기화 시각 표기 중

UI	sub 상태키	정의	

배포 상태 UI

배포 상태에 따른 상태값 변화

```mermaid
---
config:
  layout: fixed
---
flowchart LR
 subgraph S1["코드 대기 sub-state"]
        S1b["REPO_READY"]
        S1a["REPO_PROVISIONING"]
  end
 subgraph S2["점검 sub-state"]
        S2c["REVIEW_IN_PROGRESS"]
        S2b["WAITING_REVIEW"]
        S2a["PR_OPENED"]
        S2e["RESUBMITTED"]
        S2d["CHANGES_REQUESTED"]
  end
 subgraph S4["배포 중 sub-state"]
        S4a["DEPLOYING"]
        S4b["DEPLOY_FAILED"]
  end
 subgraph S5["배포 완료 sub-state"]
        S5a["DEPLOYED"]
  end
    A(["A 코드 대기"]) --> B(["B 코드 및 보안 점검"])
    C(["C 코드 승인"]) --> D(["D 배포 중"])
    D --> E(["E 배포 완료"])
    D -- 실패 --> DF["배포 요청 버튼 클릭"]
    DF -- 재시도 --> D
    S1a --> S1b
    S2a --> S2b
    S2b --> S2c
    S2c -- 반려 --> S2d
    S2d -- 수정 후 push --> S2e
    S4a -- 실패 --> S4b
    S4b -- 재시도 --> S4a
    A -.-> S1a
    B -.-> S2a
    D -.-> S4a
    E -.-> S5a
    S4a -. success .-> E
    S2c --> n1["APPROVED"]
    n1 --> C
    S2e -- 재점검 --> S2a

    DF@{ shape: lean-l}
    n1@{ shape: rect}
```

- 아직 가지못한 상태는 회색
- 각 단계가 완료되었으면 초록색
- 상태가 변경되었지만, 완료안되면 **검정색**

코드 대기	`REPO_PROVISIONING`	프로젝트 저장 직후, GitHub 레포 생성 작업이 진행 중인 상태 (아직 repo_url 미확정 가능)	또는 null 로처리
	`REPO_READY`	레포 생성 완료 및 repo_url 확보됨. 아직 코드/PR이 들어오지 않아 대기 중	
코드 및 보안 점검	`PR_OPENED`	PR이 생성되어 점검 플로우가 시작됨(최초 진입)	
	`WAITING_REVIEW`	승인자(운영자)가 PR 리뷰를 하기 전 대기 상태	
	`REVIEW_IN_PROGRESS`	리뷰 진행 중(리뷰 착수/할당/코멘트 진행 등)	
	`CHANGES_REQUESTED`	**반려(Changes requested)** 상태. 사용자(외부 작업자)가 수정 후 다시 커밋/푸시해야 함	
	`RESUBMITTED`	반려 후 새 커밋이 push되어 재점검 사이클로 돌아온 상태(이후 `WAITING_REVIEW` 로 전이)	
코드 승인	`APPROVED`	PR 리뷰 Approved로 승인 확정(배포 트리거 조건 충족)	
배포중	`DEPLOYING`	배포 진행 중(빌드/업로드/반영 단계 포함)	
	`DEPLOY_FAILED`	배포 실패(빌드 실패, 업로드 실패, CDN 반영 실패, 헬스체크 실패 등)	
배포 완료	`DEPLOYED`	배포 성공 및 서비스 가능한 상태(최종 완료)	

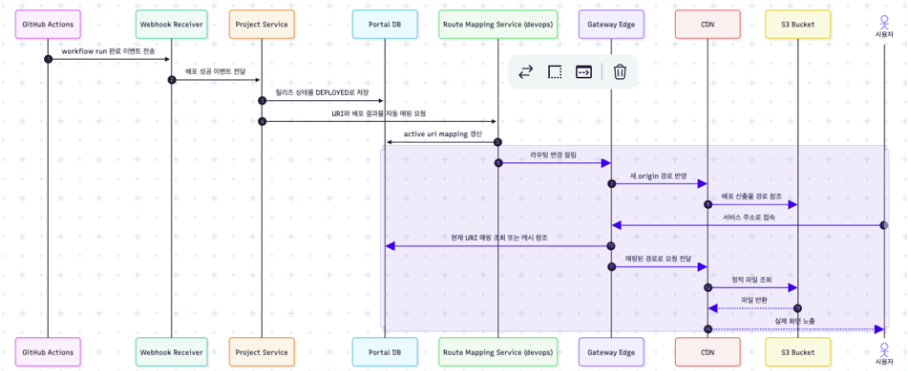
4.3. 코드 유효성/보안 검증

방식에 따라서 플로우를 그려보자~

	기능	설명	우선순위	비고

1	**스크리닝 코드**	• **liquid.js 한정 범위** ◦ 업로드된 코드가 Liquid 템플릿 규칙에 맞는지 ◦ 반드시 template.liquid와 data.json 파일이 명확한 폴더 구조 내에 존재하는지 확인합니다. ◦ {{ title }}과 같은 Liquid 문법을 준수했는지 ◦ 보안을 위협하는 코드가 없는지 검사합니다. • **방식** ◦ **미리 보안 및 형식관련한 코드를 만들어두고 동작시키는 방식** ▪ 코드 푸시 > 올라오자마자 돌게 만들고, 이것은 devops가 셋팅 > 스크리닝 코드 작동> 깃헙 PR 상태로 업데이트 > PR을 보고 외부 파트너가 태그 생성, 최종 머지> 배포 버전 선택 > 자동 배포 ▪ 포털 내 PR중인 상태를 추가할 건지는 진행하면서 필요하면 적용.		

4.4. 라우팅



라우팅

```
sequenceDiagram
  autonumber
  participant ACT as GitHub Actions
  participant WH as Webhook Receiver
  participant PS as Project Service
  participant DB as Portal DB
  participant RM as Route Mapping Service
  participant EDGE as Gateway Edge
  participant CDN as CDN
  participant S3 as S3 Bucket
  actor User as 사용자

  ACT->>WH: workflow run 완료 이벤트 전송
  WH->>PS: 배포 성공 이벤트 전달
  PS->>DB: 릴리즈 상태를 DEPLOYED로 저장
  PS->>RM: URI와 배포 결과물 자동 매핑 요청
  RM->>DB: active uri mapping 갱신
  RM->>EDGE: 라우팅 변경 알림
  EDGE->>CDN: 새 origin 경로 반영
  CDN->>S3: 배포 산출물 경로 참조

  User->>EDGE: 서비스 주소로 접속
  EDGE->>DB: 현재 URI 매핑 조회 또는 캐시 참조
```

```
  EDGE->>CDN: 매핑된 경로로 요청 전달
  CDN->>S3: 정적 파일 조회
  S3-->>CDN: 파일 반환
  CDN-->>User: 실제 화면 노출
```

1	위 그림에서 보라색 박스 영역은 메나가 구현할 범위가 아닌지 확인필요	라우팅 매핑 서비스 api를 데브옵스에서 받아서 구현하고 api, 정보스키마 주시면됨.
2	라우팅 방식 둘중 케이스 1이 맞는지?	**현재 개발될 방식. S3 파일을 여전히 쓰지만, "자동화" 되는 방식 (가장 확률이 높음)** • **방식:** 관리자가 포털 UI에서 [배포/활성화] 버튼을 누르면, 시스템(백엔드)이 데이터베이스(DB)를 업데이트함과 동시에 **S3에 있는 라우팅 설정 파일(예: `routes.json`)을 '기계가 알아서' 새로 작성하고 덮어씌웁니다.** • **기획자/운영자 입장:** 예전처럼 개발자가 AWS S3 콘솔에 직접 들어가서 파일을 열고 타이핑할 일은 0%로 사라집니다. • **결론:** 시스템 뒤편에서는 여전히 S3 파일을 사용할 수 있지만, 사람의 손을 타지 않는 자동화된 결과물(Artifact)로 쓰임새가 바뀝니다. **추후 생각하는방식** **S3 라우팅 파일을 아예 없애고, "동적 라우팅(DB/Cache 조회)"으로 넘어가는 방식** • **방식:** 포털 UI에서 라우팅 정보를 저장하면 DB(또는 Redis 같은 빠른 캐시 메모리)에 저장됩니다. 고객이 `도메인/path` 로 접속하면, 최전방의 게이트웨이(API Gateway나 CloudFront Edge)가 **S3 파일을 읽는 대신 DB/캐시를 0.001초 만에 찔러서** "아, 이 경로는 v1.2.4 태그를 보여주면 되는구나" 하고 바로 연결합니다. • **결론:** 이 경우 S3에는 '라우팅 매핑 파일'은 아예 사라지고, 순수하게 개발자가 만든 '화면 리소스(HTML, JS, 이미지 파일 등)'만 저장하는 창고 역할만 하게 됩니다.